COMPUTER NETWORKS · EXAM PREP

Transport Layer

UDP · Reliable data transfer · GBN / SR · TCP · Flow & congestion control · QUIC

Built from Chapter 3 of Kurose & Ross (8th ed., slides v8.2) and shaped around the question pattern of your previous exam: short 2-mark “why / compare” questions, 3-mark “explain / trace / trade-off” questions, and a few 3–4 mark numericals.

0	How your exam is set, and where to focus	2
1	Crash notes: every Chapter 3 topic, condensed	4
2	One-page formula sheet	11
3	Worked numericals (exam-style, fully solved)	12
4	Predicted questions with model answers (2, 3 and 4 marks)	16
5	Traps, and a last-hour checklist	24

How to use this in the time you have

- **1 day left:** Section 0 → Section 2 (formulas) → Section 3 (do each numerical yourself before reading the solution) → Section 4 → Section 5.
- **2–3 hours left:** Section 2, the ★★★ rows in Section 0, all of Section 4, then Section 5.
- **Answer length:** your paper gives ~2–3 minutes per mark. A 2-mark answer is 3–5 crisp lines or a 2-row comparison; a 3–4 mark answer needs a diagram, a table or a worked calculation.

0 · How your exam is set, and where to focus

Your last paper had 22 questions and about 50 marks. Sorted by type, the pattern is clear, and it will almost certainly repeat for the Transport Layer:

Question type in last paper	Examples (Q#)	What the Transport-layer version will look like
"Why?" design reasoning (2 marks)	Q1 asymmetry, Q3 fiber, Q22 botnets	Why a 3-way handshake and not 2? Why do DNS and video use UDP? Why do 2 sequence numbers suffice in rdt3.0? Why 3 duplicate ACKs? Why does the SR window need to be ≤ half the sequence space?
Compare / contrast (2–3 marks)	Q2, Q5, Q6, Q8, Q12, Q18	TCP vs UDP (in more depth this time), GBN vs SR, flow vs congestion control, Tahoe vs Reno, AIMD vs CUBIC vs delay-based (BBR), end-to-end vs network-assisted (ECN), TCP+TLS vs QUIC
Numerical (2–4 marks)	Q10 queueing delay, Q14 HTTP timing, Q15 DNS delay, Q19 BitTorrent	Stop-and-wait utilisation, window size for a target utilisation, sequence-number bits for GBN/SR, EWMA timeout, cwnd trace, TCP throughput (¾W/RTT), rwnd-limited throughput, checksum, seq/ACK numbers
Trace a process step by step (3–4 marks)	Q15 DNS resolution, Q13/Q17 DASH logic	3-way handshake with seq/ACK values, connection teardown and TIME_WAIT, GBN/SR with a lost packet, fast retransmit, slow start → congestion avoidance → loss
Tools and sockets (2–3 marks)	Q4 traceroute, Q16 blocking calls / listen backlog, Q20 Wireshark	Using Wireshark to see handshakes, retransmissions and window updates; netstat/ss states; demultiplexing with bind/accept; TCP vs UDP sockets; SYN and accept queues
Security (2 marks)	Q9 DNS security, Q22 DoS	SYN flood and SYN cookies, UDP amplification, TCP reset/hijack through sequence-number guessing, why QUIC encrypts its headers

> **Key observation about your professor**
>
> Several questions went **beyond the slides**: HTTP/2 multiplexing, DASH buffer logic, tit-for-tat alternatives, Wireshark limits, and the listen backlog. For this chapter, expect the same kind of extension: **QUIC / HTTP/3, CUBIC, BBR, ECN, SYN floods, Karn's algorithm, TIME_WAIT, window scaling**. All of these are covered below.

Topic priority map

Topic	Priority	Why
TCP vs UDP; mux/demux (port vs 4-tuple)	★★★	Asked last time at the application level; now expect it with header, socket and demux detail
rdt 2.0 → 3.0 (checksum, ACK/NAK, seq #, timer)	★★★	Classic "why is each mechanism needed" 2-mark material
Stop-and-wait utilisation, pipelining, GBN vs SR	★★★	The easiest numerical to set, plus a compare question
TCP seq/ACK, RTT estimation and timeout, fast retransmit	★★★	Numerical (EWMA) and trace questions
3-way handshake, closing, socket calls, SYN flood	★★★	Matches Q16 (sockets) and Q22 (DoS) from last time

Congestion control: slow start, AIMD, Tahoe/Reno, cwnd trace	★★★	The most likely 4-mark question
Flow control (rwnd) vs congestion control (cwnd)	★★☆	2-mark compare
CUBIC, delay-based/BBR, ECN, fairness, parallel connections	★★☆	Trade-off question in the style of Q5/Q21
QUIC, HTTP/3, head-of-line blocking	★★☆	Links back to Q12/Q14 (HTTP/2) from last paper
Causes and costs of congestion (3 scenarios)	★☆☆	Possible 3-mark “explain”
Internet checksum arithmetic; TCP throughput and Mathis formula	★☆☆	Short numerical

1 · Crash notes

1.1 Transport-layer services

- The transport layer provides **logical communication between processes**. The network layer (IP) provides logical communication between **hosts**. Transport relies on IP and enhances it.
- It runs **only in end systems**, not in routers. The sender breaks application messages into **segments** and passes them to IP. The receiver reassembles them and passes them up through a **socket**.
- **TCP**: reliable, in-order byte stream, flow control, congestion control, connection setup. **UDP**: unreliable, unordered, a "no-frills" extension of best-effort IP.
- **Neither** TCP nor UDP provides delay guarantees or bandwidth guarantees.

1.2 Multiplexing and demultiplexing

	UDP (connectionless demux)	**TCP (connection-oriented demux)**
Socket identified by	(dest IP, **dest port**)	**4-tuple**: src IP, src port, dest IP, dest port
Two segments from different sources to the same dest port	Go to the **same** socket	Go to **different** sockets, one per connection
Web server with N clients	1 socket	1 listening (welcome) socket + N connection sockets, all on port 80

Multiplexing (sender): gather data from many sockets and add headers with port numbers. **Demultiplexing** (receiver): use header fields to deliver each segment to the correct socket. Multiplexing and demultiplexing happen at *every* layer.

1.3 UDP [RFC 768]

- Header is **8 bytes**: src port, dst port, length (header + data, in bytes) and checksum, each 16 bits.
- No handshake, so **no RTT of setup delay**. No connection state, so a server can handle more clients. Small header. No congestion control, so the application sends at any rate it likes.
- Used by streaming media (loss-tolerant, rate-sensitive), **DNS**, SNMP, and **HTTP/3 (QUIC)**.
- If an application needs reliability over UDP, it must add reliability and congestion control at the **application layer** (this is what QUIC does).

Internet checksum

- Sender: treat the segment (plus a pseudo-header containing the IP addresses) as 16-bit words, add them in **1's-complement arithmetic** (any carry out of the top bit wraps around and is added back), then invert the result.
- Receiver: add all words *including* the checksum. The result should be all 1s; otherwise an error has been detected.
- **Weak**: if two bits in the same column flip in opposite directions (one 0→1, another 1→0), the sum is unchanged and the error goes undetected.

1.4 Principles of reliable data transfer (rdt)

Version	**Channel assumption**	**Mechanism added, and why**
rdt1.0	Perfectly reliable	Nothing needed: just send and receive

rdt2.0	Bit errors	**Checksum** (detect errors) + **ACK/NAK** (feedback) + **retransmission**. This is a stop-and-wait protocol. *Fatal flaw*: if the ACK/NAK itself is corrupted, the sender cannot tell what happened, and a blind retransmission may create a duplicate.
rdt2.1	Bit errors, including in ACK/NAK	**Sequence numbers (0/1)** so the receiver can detect and discard duplicates. Two numbers suffice because stop-and-wait has only one packet outstanding: the receiver only needs to tell "new" from "repeat of the last one".
rdt2.2	Same as 2.1	**NAK-free**: the receiver ACKs the last correctly received packet, and a **duplicate ACK acts as a NAK**. TCP uses this approach.
rdt3.0	Bit errors **and loss**	**Countdown timer**: retransmit if no ACK arrives within a "reasonable" time. If the packet was only delayed, the retransmission is a duplicate, which sequence numbers already handle. Also called the **alternating-bit protocol**.

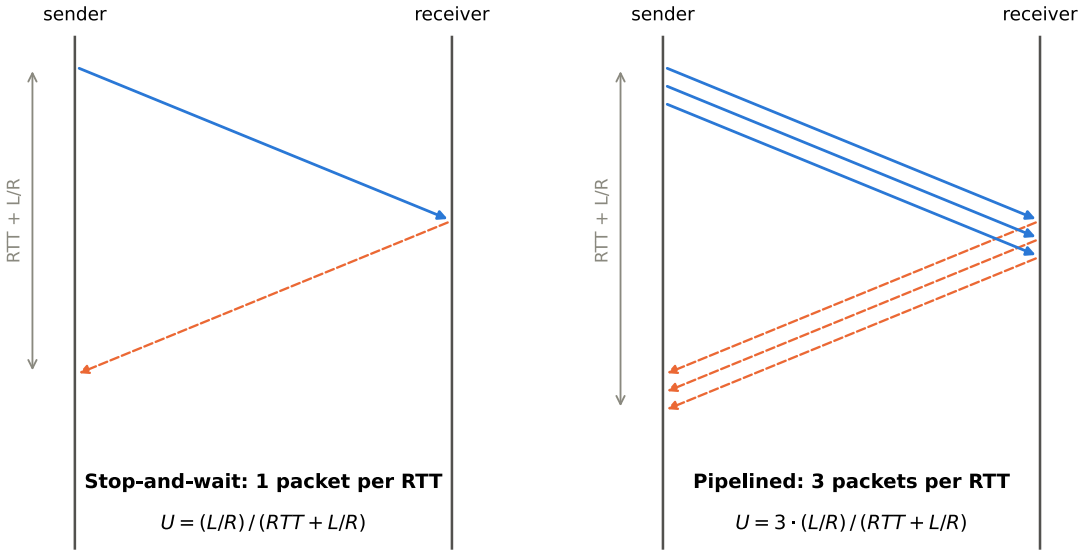


Figure 1. Stop-and-wait wastes the pipe; pipelining N packets multiplies utilisation by N (until N·L/R ≥ RTT + L/R).

rdt3.0 performance is poor. With a 1 Gbps link, 15 ms propagation delay and an 8000-bit packet: L/R = 8 µs, so U = 0.008 / 30.008 ≈ **0.00027**, and throughput is only about 267 kbps on a 1 Gbps link. *The protocol limits the performance of the underlying infrastructure*. The fix is **pipelining**, which needs a larger sequence-number range and buffering at the sender and/or receiver.

1.5 Go-Back-N vs Selective Repeat

	Go-Back-N	**Selective Repeat**
ACK type	**Cumulative**: ACK(n) acknowledges everything up to and including n	**Individual**: each correctly received packet is ACKed
Timers	**One** timer, for the oldest unACKed packet	(Conceptually) **one per** unACKed packet
On timeout	Retransmit packet n **and all higher** packets in the window	Retransmit **only** that packet
Out-of-order at receiver	Discarded (or buffered, as an implementation choice); re-ACK the highest in-order packet	**Buffered**, then delivered in order once the gap fills
Receiver state	Only `expectedseqnum`	A receive window and a buffer

Max window, k-bit seq #	$N \le 2^k - 1$	$N \le 2^{k-1}$ (half the sequence space)
Best when	Loss is rare (simple, little buffering)	Loss is frequent or BDP is large (avoids needless retransmissions)

SR dilemma (slide 3-79/80): with sequence numbers 0–3 and window 3, the receiver cannot tell whether an arriving "pkt0" is new data or a retransmission of the old pkt0 whose ACK was lost. Its view is identical in both cases. The fix is **window ≤ ½ × sequence-number space**.

1.6 TCP overview [RFCs 793, 1122, 2018, 5681, 7323]

- **Point-to-point** (one sender, one receiver), **full duplex**, a **reliable, in-order byte stream** with no message boundaries, **connection-oriented** (handshake), **pipelined** (window set by congestion and flow control), and **flow controlled**.
- **MSS** (maximum segment size) is the largest amount of *application data* in a segment. It is typically 1460 bytes, which is 1500-byte Ethernet MTU − 20 bytes IP − 20 bytes TCP.

TCP segment header (20 bytes + options)

Field	Bits	Purpose
Source port / dest port	16 + 16	Demultiplexing
Sequence number	32	Byte-stream number of the **first data byte** in this segment
Acknowledgement number	32	**Next byte expected** from the other side (cumulative)
Header length	4	In 32-bit words (options make the header variable-length)
Flags: C, E, U, A, P, R, S, F	1 each	CWR/ECE (ECN), URG, ACK valid, PSH, RST, SYN, FIN
Receive window (rwnd)	16	Flow control: free buffer bytes at the receiver
Checksum, urgent pointer	16 + 16	Error detection (Internet checksum)
Options	variable	MSS, window scale, SACK, timestamps

Sequence numbers and ACKs

- Sequence numbers count **bytes, not segments**. The ACK carries the sequence number of the next byte expected, and it is **cumulative**. **SYN and FIN each consume one sequence number.**
- How out-of-order segments are handled is **not specified**; it is left to the implementer (in practice, they are buffered).
- TCP looks like a **GBN/SR hybrid**: cumulative ACKs and a single retransmission timer (like GBN), but it buffers out-of-order data and retransmits only the *one* missing segment (like SR). SACK makes it even closer to SR.

RTT estimation and timeout

```
EstimatedRTT = (1 − α)·EstimatedRTT + α·SampleRTT      α = 0.125   (EWMA)
DevRTT       = (1 − β)·DevRTT + β·|SampleRTT − EstimatedRTT|   β = 0.25
TimeoutInterval = EstimatedRTT + 4·DevRTT
```

- A **timeout that is too short** causes premature timeouts and unnecessary retransmissions. **Too long** means slow reaction to loss.
- **SampleRTT ignores retransmitted segments** (Karn's algorithm): when an ACK arrives after a retransmission, you cannot tell whether it acknowledges the original or the retransmission, so the sample would be ambiguous.
- On a timeout, TCP **doubles** the timeout interval (exponential backoff), which is a mild form of congestion control.

TCP sender events (simplified)

1. **Data from the application**: create a segment with seq # = byte number of its first byte, and start the timer if it is not already running (the timer belongs to the oldest unACKed segment).
2. **Timeout**: retransmit the segment that caused the timeout, and restart the timer.
3. **ACK received**: if it acknowledges previously unACKed data, update SendBase, and restart the timer if unACKed segments remain.

Receiver ACK generation [RFC 5681]

Event at receiver	Action
In-order segment arrives; all earlier data already ACKed	**Delayed ACK**: wait up to 500 ms for the next segment, and ACK if none arrives
In-order segment arrives; another in-order segment has an ACK pending	Immediately send one cumulative ACK covering both
Out-of-order segment (gap detected)	Immediately send a **duplicate ACK** carrying the next expected byte
Segment fills the gap partly or fully	ACK immediately, if the segment starts at the lower end of the gap

Fast retransmit: if the sender receives **3 duplicate ACKs** for the same data, it resends the smallest unACKed segment **without waiting for the timeout**. The threshold is 3 and not 1 because *reordering* alone can produce one or two duplicate ACKs. Three duplicates are strong evidence of real loss, and they also show that later segments are still getting through, so the network is not badly congested.

Classic retransmission scenarios: (1) *Lost ACK*: a timeout causes a retransmission, and the receiver discards the duplicate and re-ACKs. (2) *Premature timeout*: the retransmission is unneeded, but a cumulative ACK soon covers it. (3) *Cumulative ACK covers an earlier lost ACK*: ACK 120 arrives before the timer fires, so nothing is resent.

1.7 TCP flow control

- Problem: the network delivers data faster than the application reads it from the socket buffer, so the **receiver's buffer would overflow**.
- The receiver advertises free space in the **rwnd** field: `rwnd = RcvBuffer − (LastByteRcvd − LastByteRead)`.
- The sender keeps `LastByteSent − LastByteAcked ≤ rwnd`, which guarantees no overflow. RcvBuffer is set through socket options (a typical default is 4096 bytes), and many operating systems auto-tune it.
- **Zero-window case:** when rwnd = 0, the sender keeps sending 1-byte *probe* segments (persist timer) so that it learns when space frees up. Otherwise the two sides could deadlock.
- With the 16-bit rwnd field the maximum window is 64 KB, so the **window scale option** (RFC 7323) is needed on high bandwidth-delay-product paths.

1.8 TCP connection management



Figure 2. 3-way handshake, data transfer and 4-segment close, with TCP states.

- **Why a 2-way handshake fails:** with variable delays, retransmitted requests, reordering, and the fact that neither side can "see" the other, an **old duplicate connection request** can arrive after the client has finished. The server then opens a **half-open connection** with no client, or accepts **duplicate old data** as new. The third message lets the **client confirm** that the server's SYNACK belongs to a live request, and both sides agree on **initial sequence numbers**.
- Socket calls: the server calls `socket()` → `bind()` → `listen(backlog)` → `accept()`, and accept *blocks* and then returns a **new connection socket**. The client calls `socket()` → `connect()`, which sends the SYN and blocks until the handshake completes.
- **Closing:** each side closes its own direction by sending a **FIN**, and the other side ACKs it. The ACK can be combined with the ACK-sender's own FIN, and simultaneous FINs are handled. The side that closes first waits in **TIME_WAIT (2 × MSL)** so that (a) it can re-ACK if the final ACK is lost and the peer's FIN is retransmitted, and (b) stray old segments die out before the same 4-tuple is reused.

1.9 Principles of congestion control

Congestion means "too many sources sending too much data too fast for the *network* to handle". It shows up as **long delays** (queueing in router buffers) and **packet loss** (buffer overflow). It is **different from flow control**, which is about one sender overwhelming one receiver.

Scenario	Setup	Cost of congestion revealed
1	2 flows, 1 router, **infinite** buffers, link R	Throughput per connection is capped at R/2, and **delay grows without bound** as the arrival rate approaches R/2
2	1 router, **finite** buffers, retransmissions	**More work (retransmissions)** for the same goodput; **unneeded duplicates** from premature timeouts waste link capacity, so maximum achievable throughput falls
3	4 senders, **multi-hop** paths	A packet dropped downstream **wastes all the upstream capacity and buffering** used to carry it; throughput can collapse toward 0

Two approaches: end-to-end (no explicit feedback from the network; congestion is inferred from loss or delay; *this is TCP's approach*) and **network-assisted** (routers give direct feedback, which may indicate congestion or set a rate; examples are TCP ECN, ATM and DECbit).

1.10 TCP congestion control

- The sender limits data in flight: `LastByteSent − LastByteAcked ≤ min(cwnd, rwnd)`, so the rate is roughly **cwnd / RTT** bytes per second.
- **AIMD**: *additive increase* of cwnd by 1 MSS every RTT until loss, then *multiplicative decrease* (cut in half) on loss. The result is the **sawtooth** shape, which probes for bandwidth. AIMD is a distributed, asynchronous algorithm that optimises congested flow rates network-wide and is stable.
- **Slow start**: cwnd starts at 1 MSS and **doubles every RTT** (cwnd += 1 MSS per ACK) until the first loss or until cwnd reaches **ssthresh**. It starts slow but ramps up exponentially.
- **Congestion avoidance**: once cwnd ≥ ssthresh, grow linearly (+1 MSS per RTT).
- **On any loss**: ssthresh = cwnd / 2.
  • **Timeout** (both Tahoe and Reno): cwnd = 1 MSS, then slow start. This is a severe signal: nothing is getting through.
  • **3 duplicate ACKs**: **Tahoe** sets cwnd = 1 MSS. **Reno** halves cwnd (cwnd = ssthresh, strictly ssthresh + 3 MSS during *fast recovery*) and continues linearly. This is a milder signal: later segments are still being delivered.

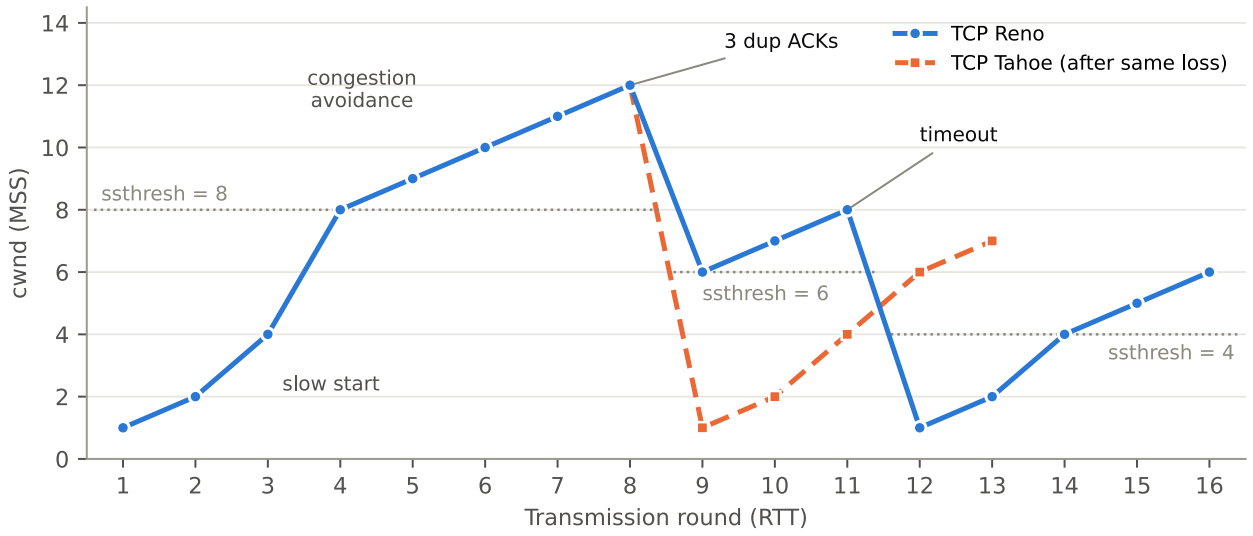


Figure 3. cwnd evolution (the same trace is solved as Numerical N6). Reno halves on 3 duplicate ACKs; Tahoe and a timeout both drop cwnd to 1 MSS.

Variant	Idea	Strength / weakness
Tahoe / Reno	Loss-based AIMD as above	Simple and fair, but slow to fill high-BDP pipes, and it fills buffers (bufferbloat)
CUBIC (Linux default)	After loss, remember W_{max} (the window at which loss happened). Grow as a **cubic function of the time since loss**: fast when far from W_{max}, cautious near the point K where W reaches W_{max}, then probe beyond it	Better throughput on long, fat pipes; growth does not depend on RTT, so it is fairer across RTTs. Still loss-based
Delay-based (Vegas, **BBR**)	Uncongested throughput = cwnd/RTT_{min}. If measured throughput is close to this, increase cwnd linearly; if it is far below, decrease linearly. "Keep the pipe just full, but no fuller"	High throughput with **low delay**, and no induced loss. Can lose out when competing with loss-based flows. BBR runs on Google's backbone

ECN (network-assisted)	A router marks 2 ECN bits in the IP header (ToS field) when congested. The receiver sets **ECE** in its TCP ACK, the sender reduces cwnd and sets **CWR**	Signals congestion **before** loss. Needs router and end-host support; the marking policy is chosen by the operator

Fairness

- Goal: K TCP sessions sharing a bottleneck of rate R each get **R/K**.
- AIMD converges to fairness. Additive increase moves both flows along a slope-1 line; multiplicative decrease halves both, pulling the point toward the equal-share line. The repeated cycles converge to R/2 each (for equal RTTs).
- **Unfairness**: (1) **UDP** flows have no congestion control and there is no “Internet police”, so they can crowd out TCP. (2) **Parallel TCP connections**: with 9 existing connections on a link of rate R, a new application using 1 connection gets R/10, but one using 11 connections gets 11/20·R ≈ R/2.

1.11 Evolution: QUIC (the basis of HTTP/3)

- QUIC is an **application-layer protocol running over UDP**. It moves transport functions (reliability, congestion control, connection setup) into user space, which makes it easy to deploy and upgrade. It is deployed in Chrome, the mobile YouTube app and Google’s servers.
- **Connection establishment**: TCP + TLS need **2 serial handshakes**. QUIC sets up reliability, congestion control, authentication and encryption state in **1 RTT** (0-RTT when resuming a connection).
- **Streams**: many application-level streams are multiplexed over one QUIC connection, each with its own reliable delivery and a **common congestion control**. A loss in one stream **does not block the others**, so there is **no head-of-line (HOL) blocking**. HTTP/2 over TCP suffers HOL blocking because TCP delivers one ordered byte stream.
- Its loss detection and congestion control parallel TCP’s (ACKs, timers, windows).

2 · Formula sheet

Transmission delay	`d_trans = L / R`
Stop-and-wait utilisation	`U = (L/R) / (RTT + L/R)`
Pipelined (window N) utilisation	`U = N·(L/R) / (RTT + L/R)` (capped at 1)
Window for 100% utilisation	`N ≥ (RTT + L/R) / (L/R) ≈ 1 + RTT·R/L`
Effective throughput	`U × R = (bits per window) / (RTT + L/R)`
GBN window, k-bit seq #	`N ≤ 2^k − 1`
SR window, k-bit seq #	`N ≤ 2^(k−1)`
Bandwidth-delay product	`BDP = R × RTT (bits "in the pipe")`
Window-limited throughput	`Throughput ≤ min(cwnd, rwnd) / RTT`
rwnd	`rwnd = RcvBuffer − (LastByteRcvd − LastByteRead)`
EWMA RTT	`Est = 0.875·Est + 0.125·Sample`
Deviation	`Dev = 0.75·Dev + 0.25·|Sample − Est|`
Timeout	`TimeoutInterval = Est + 4·Dev`
Avg TCP throughput (AIMD, no slow start)	`≈ 0.75 · W / RTT (W = window at loss)`
Mathis formula (loss prob. L)	`Throughput ≈ 1.22 · MSS / (RTT · √L)`
Slow start: segments sent in n RTTs	`1 + 2 + 4 + … + 2^(n−1) = 2^n − 1`
Fair share	`R / K for K flows; a flow with m of M total connections gets m·R/M`
Seq # wrap-around	`2^32 bytes ≈ 4.29 GB; time = 2^32 × 8 / R`
Header sizes	`UDP 8 B · TCP 20 B (+options) · IPv4 20 B · MSS = MTU − 40`

Unit hygiene (where marks are lost)

- Bytes × 8 = bits. Link rates are in **bits**/s, and buffers and files are usually in **bytes**.
- 1 Mbps = 10^6 bits/s. For files, state whether 1 MB = 10^6 or 2^{20} bytes. Either is fine if you say which.
- RTT = 2 × propagation delay (if the question gives one-way delay). Some questions ignore L/R; say so if you do.
- cwnd and ssthresh are in MSS in trace questions and in bytes in throughput questions.

TCP congestion control in 6 lines

```
start:           cwnd = 1 MSS, ssthresh = large (or given)
cwnd < ssthresh: slow start,     cwnd doubles per RTT (+1 MSS per ACK)
cwnd ≥ ssthresh: cong. avoidance, cwnd += 1 MSS per RTT
3 dup ACKs:      ssthresh = cwnd/2; Reno: cwnd = ssthresh (+3); Tahoe: cwnd = 1
timeout:         ssthresh = cwnd/2; cwnd = 1 MSS  (both); timeout doubles
sending window = min(cwnd, rwnd)
```

3 · Worked numericals

Cover each solution, try the problem yourself, then check. These cover every calculation that can reasonably be set from Chapter 3.

N1 **Stop-and-wait utilisation (slide 3-67)**

A 1 Gbps link has a 15 ms one-way propagation delay and 8000-bit packets. Find (a) the sender utilisation of rdt3.0, (b) its effective throughput, (c) the utilisation with 3 packets pipelined, and (d) the window size needed for 100% utilisation.

L/R = 8000 / 10^9 = 8 µs = 0.008 ms. RTT = 2 × 15 = 30 ms.
(a) U = 0.008 / (30 + 0.008) = **0.00027** (0.027%).
(b) Throughput = 8000 bits / 30.008 ms ≈ **267 kbps** on a 1 Gbps link.
(c) U = 3 × 0.008 / 30.008 = **0.0008** (3× better).
(d) N ≥ 30.008 / 0.008 ≈ **3751 packets** in flight.

N2 **Window size and sequence-number bits**

R = 100 Mbps, packet size 1500 bytes, RTT = 40 ms. (a) What is the stop-and-wait utilisation? (b) What window size gives at least 90% utilisation? (c) How many sequence-number bits are needed for GBN, and for SR, with that window?

L = 1500 × 8 = 12,000 bits, so L/R = 12,000 / 10^8 = 0.12 ms.
(a) U = 0.12 / 40.12 = **0.30%**.
(b) N × 0.12 / 40.12 ≥ 0.9 gives N ≥ 300.9, so **N = 301**.
(c) GBN: $2^k - 1 \geq 301$, so k = **9 bits** (511). SR: $2^{k-1} \geq 301$, so $2^9 = 512$ and k = **10 bits**.

N3 **Internet checksum (slide 3-38)**

Compute the checksum of the two 16-bit words 1110011001100110 and 1101010101010101.

```
  1110 0110 0110 0110
+ 1101 0101 0101 0101
= 1 1011 1011 1011 1011   (17 bits: carry out)
wraparound: 1011 1011 1011 1011 + 1 = 1011 1011 1011 1100   (sum)
checksum = 1's complement  =  0100 0100 0100 0011
```

The receiver adds both words and the checksum. The result is 1111 1111 1111 1111 if there is no error. If the last bit of both data words flips (0→1 and 1→0), the sum is unchanged, so the error goes **undetected**. This is why the checksum is called weak.

N4 **TCP sequence and ACK numbers**

Host A sends two back-to-back TCP segments to B. The first has seq = 127 and 80 bytes of data; the second has 40 bytes. A's port is 302 and B's port is 80. (a) What are the second segment's seq number and ports? (b) If the first arrives first, what are the ACK number and ports in B's ACK? (c) If the second arrives first, what is the ACK number? (d) If the first is lost and the second arrives, what does B send, and what does A do after 3 such ACKs?

(a) seq = 127 + 80 = **207**, src port 302, dst port 80.
(b) ACK = **207** (next byte expected), src port 80, dst port 302.
(c) ACK = **127**: B is still waiting for byte 127. This is a duplicate ACK, and B buffers the out-of-order segment.
(d) B keeps sending ACK 127 for each later segment. After 3 duplicate ACKs, A **fast-retransmits** the segment starting at 127 without waiting for the timeout. B then sends a cumulative ACK of 127 + 80 + 40 = **247** (plus any later bytes it has buffered).

N5 RTT estimation and timeout

EstimatedRTT = 100 ms and DevRTT = 10 ms. A new SampleRTT of 120 ms arrives. With α = 0.125 and β = 0.25, find the new EstimatedRTT, DevRTT and TimeoutInterval.

Est = 0.875 × 100 + 0.125 × 120 = 87.5 + 15 = **102.5 ms**
Dev = 0.75 × 10 + 0.25 × |120 − 102.5| = 7.5 + 4.375 = **11.875 ms**
Timeout = 102.5 + 4 × 11.875 = **150 ms**
(RFC 6298 updates DevRTT using the *old* EstimatedRTT: 0.75×10 + 0.25×20 = 12.5 ms, giving Timeout = 152.5 ms. State which order you use; both are accepted if you are consistent.)

N6 cwnd trace (see Figure 3)

TCP Reno starts with cwnd = 1 MSS and ssthresh = 8. The window grows each round with no loss until round 8 ends with 3 duplicate ACKs. Round 11 ends in a timeout. Give cwnd for rounds 1–16, the slow-start and congestion-avoidance intervals, and the ssthresh values. What would Tahoe do after round 8?

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
cwnd	1	2	4	8	9	10	11	12	6	7	8	1	2	4	5	6

Slow start: rounds 1–4 and 12–14. **Congestion avoidance**: rounds 4–8, 9–11 and 14–16.
ssthresh = 8 initially. After the 3 duplicate ACKs at cwnd = 12, ssthresh = **6** and cwnd = 6. After the timeout at cwnd = 8, ssthresh = **4** and cwnd = 1.
Tahoe after round 8: ssthresh = 6 but cwnd = 1, giving 1, 2, 4, 6 (capped at ssthresh), 7, …
How to read a K&R-style graph: exponential growth means slow start; a straight +1 line means congestion avoidance; a drop to half means 3 duplicate ACKs; a drop to 1 means timeout (or Tahoe).

N7 Slow start and object download time

A client downloads a 15-MSS object. RTT = 100 ms; ignore transmission time, and assume the object size is below rwnd and no loss occurs. How long does it take from the start of the TCP connection, (a) with an initial cwnd of 1 MSS, and (b) with the modern initial window of 10 MSS?

(a) Handshake: 1 RTT. Then the server sends windows of 1, 2, 4 and 8 segments (1+2+4+8 = 15), which takes 4 RTTs. Total ≈ **5 RTT = 500 ms**.
(b) Handshake 1 RTT, then 10 segments, then 5 segments, which takes 2 RTTs. Total ≈ **3 RTT = 300 ms**.
Takeaway: for short web transfers, **latency (RTT count) dominates, not bandwidth**. This is why TCP Fast Open, larger initial windows and QUIC's 1-RTT/0-RTT setup matter.

N8 Average TCP throughput and long fat pipes (slides 3-153/154)

(a) A TCP connection has W = 20 segments of 1500 B at loss, and RTT = 100 ms. What is its average throughput? (b) To reach 10 Gbps with 1500 B segments and a 100 ms RTT, how many segments must be in flight, and what loss rate can TCP tolerate?

(a) Average = 0.75 × W / RTT = 0.75 × (20 × 1500 × 8) / 0.1 = **1.8 Mbps**.
(b) W = 10^{10} × 0.1 / 12,000 ≈ **83,333 segments**. From the Mathis formula, L = (1.22 × MSS / (RTT × throughput))² = (1.22 × 12,000 / (0.1 × 10^{10}))² ≈ **2×10^{-10}**. This is an unrealistically small loss rate, which is why variants for high-speed paths (CUBIC, BBR) were developed.

N9 Flow control, rwnd and BDP

(a) RcvBuffer = 4096 B, LastByteRcvd = 3000 and LastByteRead = 1000. What is rwnd? (b) The receiver advertises the maximum 16-bit window (65,535 B) and RTT = 100 ms. What is the maximum throughput? (c) What window is needed to fill a 1 Gbps path with RTT = 100 ms?

(a) rwnd = 4096 − (3000 − 1000) = **2096 bytes**.
(b) Throughput ≤ rwnd / RTT = 65,535 × 8 / 0.1 ≈ **5.24 Mbps**, however fast the link.
(c) BDP = $10^9 \times 0.1 = 10^8$ bits = **12.5 MB**. This is far above 64 KB, so the **window scale option** (RFC 7323) is required.

N10 Fairness with parallel connections (slide 3-139)

A bottleneck link of rate R carries 9 TCP connections. A new application opens (a) 1 connection or (b) 11 connections. What share does it get?

(a) 1/10 × R = **R/10**. (b) 11/(9 + 11) × R = 11/20 × R ≈ **R/2**. TCP is fair *per connection*, not per application or per user.

N11 GBN vs SR transmission count

The window is N = 4 and packets 0–7 are to be sent. Packet 2 is lost once; everything else arrives. How many transmissions does each protocol need?

GBN: send 0–3 (2 is lost). ACK0 and ACK1 slide the window, so send 4 and 5. The receiver discards 3, 4 and 5 and re-ACKs 1. The timer for 2 expires, so resend **2, 3, 4 and 5**, then send 6 and 7. Total = 8 + 4 = **12**.
SR: the receiver buffers 3, 4 and 5 and ACKs each one. Only 2 times out and is resent. Total = 8 + 1 = **9**.
GBN wastes bandwidth when the bandwidth-delay product is large and loss occurs.

N12 Sequence-number wrap-around

(a) What is the largest file that can be sent without the 32-bit TCP sequence number wrapping? (b) How long does it take to wrap at 10 Gbps? (c) Classic K&R version: using MSS = 536 B and 66 B of headers per segment on a 155 Mbps link, how long does sending the maximum file take?

(a) 2^{32} bytes ≈ **4.29 GB** (sequence numbers count bytes).
(b) $2^{32} \times 8 / 10^{10}$ ≈ **3.4 s**. That is shorter than a segment's possible lifetime in the network, so old segments could be confused with new ones. The fix is **PAWS** (timestamps option).
(c) Segments = $\lceil 2^{32}/536 \rceil$ = 8,012,999. Total bytes = 2^{32} + 8,012,999 × 66 ≈ 4.824×10^9 B. Time = $4.824 \times 10^9 \times 8 / 155 \times 10^6$ ≈ **249 s**.

N13 Header overhead: UDP vs TCP for VoIP

A VoIP packet carries 100 B of payload. Compare the efficiency over UDP and over TCP (IPv4, no options).

UDP: 100 / (100 + 8 + 20) = **78%**. TCP: 100 / (100 + 20 + 20) = **71%**. For real-time traffic, TCP also adds retransmission delay and congestion-control throttling, both of which are harmful for voice, so UDP (with RTP) is preferred.

N14 **Connection setup latency: TCP+TLS vs QUIC**

RTT = 80 ms. How long does it take to receive the first response byte of a small HTTPS object with (a) TCP + TLS 1.2, (b) TCP + TLS 1.3, (c) QUIC with a new connection, and (d) QUIC 0-RTT resumption?

(a) TCP 1 RTT + TLS 1.2 2 RTT + HTTP request 1 RTT = 4 RTT = **320 ms**.
(b) 1 + 1 + 1 = 3 RTT = **240 ms**.
(c) QUIC combines the transport and crypto handshakes: 1 + 1 = 2 RTT = **160 ms**.
(d) The request is sent with the first packet: 1 RTT = **80 ms**.
This mirrors Q14 from your last paper: count round trips carefully and say what you are ignoring.

4 · Predicted questions with model answers

Written at the length the marks deserve. The small grey line under each answer notes what the examiner is testing, or which part of your last paper it mirrors.

2-mark questions

2 Why do DNS and real-time video use UDP even though it is unreliable?

- **No handshake**: a DNS query/response takes 1 RTT instead of 2, and there is no per-connection state, so a server can handle millions of clients.
- **No congestion control or retransmission delay**: video and voice are loss-tolerant but delay- and rate-sensitive. A late packet is useless, and TCP's throttling would cause stalls.
- The **header is small** (8 B vs 20 B), and the application can add only the reliability it needs (DNS retries at the application layer; QUIC implements full reliability).

Mirrors Q8 (TCP vs UDP) from your last paper; now expect the "why".

2 Why is a sequence number needed in rdt2.1, and why are two (0, 1) enough?

If an ACK/NAK is corrupted, the sender must retransmit, but the receiver may already have the packet, so it would accept a **duplicate** as new data. A sequence number lets the receiver detect and discard duplicates. In stop-and-wait, only **one packet is outstanding**, so the receiver only has to distinguish "the next packet" from "a retransmission of the previous one". One bit, alternating 0/1, is enough.

2 Why does rdt3.0 need a timer when rdt2.2 does not?

rdt2.x assumes packets may be *corrupted* but never *lost*, so some response always comes back. In rdt3.0, a data packet or ACK can be **lost**, and the sender would then wait forever. A countdown timer triggers a retransmission after a "reasonable" time. If the packet was only delayed, the retransmission creates a duplicate, which the sequence numbers already handle.

2 Differentiate flow control and congestion control.

	Flow control	Congestion control
Protects	The **receiver's buffer**	The **network** (router buffers and links)
Scope	One sender and one receiver	All senders sharing a bottleneck
Mechanism	**rwnd**, advertised explicitly in the TCP header	**cwnd**, inferred from loss or delay (or ECN)

The sender uses window = min(rwnd, cwnd).

2 How does demultiplexing differ between UDP and TCP? How many sockets does a web server with 100 active clients have?

UDP demultiplexes on the **destination port only**, so datagrams from different sources to port 53 all reach one socket. TCP uses the **4-tuple** (src IP, src port, dst IP, dst port), so each client connection gets its own socket even though all of them use dst port 80. A TCP web server therefore has **1 listening socket + 100 connection sockets = 101**, all bound to port 80.

2 What is TCP fast retransmit? Why does it wait for 3 duplicate ACKs instead of 1?

When the sender receives 3 duplicate ACKs for the same data, it retransmits the missing segment **immediately, without waiting for the timeout** (timeouts are often long). One or two duplicate ACKs can be caused by simple **reordering** in the network, so reacting to them would cause needless retransmissions. Three duplicates strongly suggest real loss, while showing that later segments are still arriving, so Reno halves cwnd instead of resetting it to 1.

2 Why is the timeout interval not simply set to the latest RTT? Why are retransmitted segments excluded from SampleRTT?

RTT fluctuates with queueing. Using a single sample would give a timeout that is **too short** (premature, unnecessary retransmissions) or too erratic. TCP therefore uses an EWMA (EstimatedRTT) plus a safety margin of 4 × DevRTT that widens when variance is high. Retransmitted segments are excluded (**Karn's algorithm**) because the ACK is **ambiguous**: it may belong to the original transmission or to the retransmission, which would corrupt the estimate.

2 Why does the Selective Repeat window have to be at most half the sequence-number space?

If the window is larger, then after all ACKs of one window are lost, the sender retransmits old packet 0. The receiver, whose window has already moved forward, also expects a *new* packet 0. Its view is **identical** in both cases (the slide 3-80 dilemma), so it accepts old data as new. Keeping $N \leq 2^{k-1}$ ensures that the old and new windows never share sequence numbers.

2 Why is a 2-way handshake insufficient for TCP?

The network can delay, duplicate and reorder messages, and neither side can see the other. An **old, retransmitted connection request** can reach the server after the client has already finished. With only 2 messages, the server would open a **half-open connection** with no client, or accept **stale duplicate data** as new. The third message (the client's ACK of the server's ISN y) proves that the client is alive and wants *this* connection. It also synchronises both initial sequence numbers.

2 What is the purpose of the TIME_WAIT state?

The side that closes first waits **2 × MSL** after sending the final ACK. (1) If that ACK is lost, the peer retransmits its FIN, and the socket still exists to re-ACK it. (2) Delayed segments from this connection can die out before the same 4-tuple is reused by a new connection, so they cannot be misread as new data.

2 What is a SYN flood attack, and how do SYN cookies defend against it?

The attacker sends many **SYNs with spoofed source addresses** and never completes the handshake. The server allocates state for each half-open connection in its **SYN (backlog) queue**, which fills up, so legitimate clients are refused. This is a DoS. With **SYN cookies**, the server allocates **no state** on receiving a SYN. It encodes the connection information in its initial sequence number (a hash of the 4-tuple, a secret and time). Only when a valid ACK returns with ack = cookie + 1 does it create the connection.

Mirrors Q22 (DoS) and Q16 (listen backlog) from your last paper.

2 Why was QUIC built on top of UDP rather than as a change to TCP?

TCP lives in the **OS kernel**, and **middleboxes** (NATs, firewalls) inspect and often block unfamiliar TCP options or new protocol numbers ("ossification"). UDP passes through almost everywhere, and QUIC runs in **user space**, so it can be deployed and updated with the browser or app. QUIC then re-implements reliability, congestion control and a 1-RTT secure handshake itself, and adds **streams without head-of-line blocking**.

2 What is head-of-line (HOL) blocking, and how does QUIC avoid it?

HTTP/2 multiplexes many objects over **one TCP byte stream**. If one TCP segment is lost, TCP will not deliver *any* later bytes, including those belonging to other objects, until the retransmission arrives, so every object stalls. QUIC gives each stream **its own reliable, in-order delivery** over UDP. A loss stalls only the affected stream, and the others continue.

Links to Q12 (HTTP/2 multiplexing) from your last paper.

2 Why is the Internet checksum called weak? Give an example.

It is a 1's-complement sum, so errors that **cancel in the sum** go undetected. For example, if bit 0 of one word flips 0→1 and bit 0 of another word flips 1→0, the sum is unchanged. Reordering of 16-bit words is not detected either. Link layers therefore use a stronger **CRC**, and TCP/UDP checksums are only a last end-to-end check.

2 Why is UDP traffic considered a threat to TCP fairness?

UDP has **no congestion control**, so a UDP application sends at a constant rate regardless of loss. When the bottleneck congests, TCP flows back off (AIMD) while UDP does not, so UDP takes more than its fair share, and heavy UDP use can push the network toward congestion collapse. There is no "Internet police" to enforce congestion control. QUIC and modern real-time protocols therefore implement congestion control on top of UDP.

2 What are the limitations of Wireshark when analysing QUIC, compared with TCP?

For TCP, even over HTTPS, Wireshark sees the **headers in cleartext**: seq/ACK numbers, flags, window and options. You can therefore reconstruct handshakes, retransmissions, RTT and window behaviour. QUIC **encrypts almost all of its transport header** (packet numbers, ACK frames, stream IDs), so without the session keys (for example an SSLKEYLOGFILE) you see only UDP port 443, sizes and timing. Congestion and loss cannot be analysed directly.

Direct extension of Q20 from your last paper.

2 Why does TCP use delayed ACKs, and what is the trade-off?

The receiver waits up to 500 ms (typically about 200 ms) before ACKing an in-order segment, so it can ACK two segments with one ACK or piggyback the ACK on reverse data. This **halves the ACK traffic** and reduces overhead. The cost is a **slightly slower cwnd growth** (cwnd grows per ACK) and more RTT samples inflated by the delay.

2 Why is slow start called "slow" if it grows exponentially?

It **starts** slow, with only 1 MSS (or a small initial window) per RTT, compared with earlier TCPs that immediately sent a full receiver window, which caused congestion collapse in 1986. From there cwnd doubles each RTT, so it discovers the available bandwidth in about $\log_2(\text{BDP/MSS})$ RTTs.

3-mark questions

3 **Explain how TCP provides reliable data transfer over the unreliable IP service. Relate each mechanism to the rdt protocols.**

Problem	rdt mechanism	How TCP does it
Bit errors	Checksum (rdt2.0)	16-bit Internet checksum over the header, data and pseudo-header
Receiver feedback	ACK / NAK → ACK-only (rdt2.2)	Cumulative ACK = next byte expected; no NAKs, since a duplicate ACK acts as a NAK
Duplicates	Sequence numbers (rdt2.1)	32-bit byte-numbered sequence numbers
Loss	Timer + retransmit (rdt3.0)	A single timer for the oldest unACKed segment, adaptive timeout, and fast retransmit on 3 duplicate ACKs
Throughput	Pipelining (GBN/SR)	Window = min(cwnd, rwnd); out-of-order segments buffered (SR-like)
Ordering	Receiver buffer	Bytes delivered to the socket strictly in order

3 **Compare Go-Back-N and Selective Repeat. Which would you choose for a satellite link with a 5% loss rate?**

Use the GBN/SR table in §1.5 (ACK type, timers, retransmission on timeout, receiver buffering, window limit), and draw a quick 4-packet timeline with pkt2 lost to show GBN resending 2–5 while SR resends only 2.
Choice: Selective Repeat. A satellite link has a large RTT, so the window (and the number of packets in flight) is large. With 5% loss, GBN would retransmit a whole window for every loss, wasting most of the capacity. SR retransmits only the lost packets, at the cost of receiver buffering and a sequence space at least twice the window size.

3 **Explain TCP congestion control: slow start, congestion avoidance, and the reaction to loss in Tahoe and Reno. Why are a timeout and 3 duplicate ACKs treated differently?**

Draw Figure 3. **Slow start**: cwnd starts at 1 MSS and doubles every RTT until it reaches ssthresh. **Congestion avoidance**: +1 MSS per RTT. **On loss**: ssthresh = cwnd/2. **Timeout**: cwnd = 1 in both variants. **3 duplicate ACKs**: Tahoe sets cwnd = 1; Reno sets cwnd = ssthresh (+3 MSS, fast recovery) and grows linearly.
Why different: a timeout means *nothing* is getting through (possibly severe congestion), so TCP restarts cautiously. Duplicate ACKs mean later segments *are* being delivered, so the path still works and only one packet was lost. Halving is enough, and it avoids wasting RTTs in slow start.

3 **Explain AIMD and show why it leads to fairness between two TCP flows.**

AIMD: **additive increase** of +1 MSS per RTT and **multiplicative decrease** (halving) on loss. Plot connection 1's throughput against connection 2's. The line x + y = R is full utilisation and the line x = y is equal share. During increase, both gain equally, so the operating point moves along a 45° line. When the total exceeds R, loss occurs and both halve, so the point moves toward the origin, which **reduces the difference** between the two by half. Repeating the cycle converges to the equal-share point R/2 each. Caveats: this assumes equal RTTs and the same MSS; a flow with a shorter RTT grows faster and gets more, and UDP or parallel connections break fairness.

3 Compare loss-based (Reno/CUBIC), delay-based (BBR) and network-assisted (ECN) congestion control in terms of throughput, delay and deployability.

	Loss-based (Reno, CUBIC)	Delay-based (Vegas, BBR)	ECN
Signal	Packet loss	RTT increase / throughput below cwnd/RTT_{min}	Router marks IP ECN bits; receiver echoes ECE
Throughput	High, and CUBIC fills long fat pipes well	High, keeping the pipe “just full”	High, reacting before loss
Delay	**High**: fills buffers until they overflow (bufferbloat)	**Low**: avoids standing queues	Low, since the router controls its queue
Deployability	Sender-only change; the universal default	Sender-only, but may lose out against loss-based flows	Needs routers, both hosts and middleboxes to cooperate

Same style as Q5 and Q21 (trade-offs, “which is more effective?”) from your last paper.

3 Explain TCP CUBIC. Why is it better than Reno on high bandwidth-delay-product paths?

Reno adds only 1 MSS per RTT, so after halving from, say, W = 83,000 segments it needs about 41,000 RTTs (over an hour at 100 ms) to recover. CUBIC records $\mathbf{W_{max}}$, the window at which loss occurred, on the assumption that the bottleneck has not changed much. It then sets $W(t) = C(t - K)^3 + W_{max}$, where K is the time at which W returns to W_{max}. Growth is **fast when far from $\mathbf{W_{max}}$, cautious near it** (plateau), and then accelerates again to probe for new bandwidth. Growth depends on **time, not on RTT**, so it is also fairer to long-RTT flows. It is the Linux default and the most widely used TCP on popular web servers.

3 Explain the causes and costs of congestion using the three textbook scenarios.

(1) With **infinite buffers**, throughput is capped at R/2 per flow, but **queueing delay explodes** as the load approaches capacity. (2) With **finite buffers**, packets are dropped and must be retransmitted, so the sender does **more work for the same goodput**. Premature timeouts send **unneeded duplicates** that waste link capacity, lowering maximum throughput. (3) With **multi-hop paths**, a packet dropped at a later hop wastes **all the upstream transmission capacity and buffering** used for it. As the load rises, goodput can **collapse toward zero** (congestion collapse). Hence the need for senders to slow down.

3 Describe the TCP 3-way handshake with sequence numbers and the corresponding socket calls. What is the role of the listen backlog?

Draw Figure 2 (top). The server calls `socket`, `bind(port)` and `listen(backlog)`, entering LISTEN, and then `accept()` **blocks**. The client’s `connect()` sends **SYN, seq = x** (SYN_SENT). The server replies **SYNACK, seq = y, ack = x + 1** (SYN_RCVD). The client sends **ACK, ack = y + 1** and may carry data (ESTAB). `connect()` returns on the client, and on the server `accept()` returns a **new connection socket** identified by the 4-tuple.
Backlog: the kernel keeps a SYN queue of half-open connections and an **accept queue** of completed connections that the application has not yet `accept()`ed. The backlog bounds the accept queue. When it is full, new SYNs are dropped or refused, which absorbs bursts but is the target of SYN floods.

3 How would you use Wireshark (or tcpdump) to study a TCP connection’s behaviour?

- **Handshake and teardown**: filter `tcp.flags.syn==1` / `tcp.flags.fin==1`. Read the ISNs, the MSS and window-scale options, and the time from SYN to SYNACK, which is the RTT.
- **RTT and throughput**: Statistics → TCP Stream Graphs (Round Trip Time, Throughput, **Stevens / tcptrace**, which plots seq # vs time; the slope is the rate and the staircase shape shows slow start).
- **Loss recovery**: `tcp.analysis.retransmission`, `tcp.analysis.duplicate_ack` and `tcp.analysis.fast_retransmission` show timeouts vs fast retransmit.
- **Flow control**: the window field, `tcp.analysis.zero_window` and window updates.

Limitation: you see only one point on the path. cwnd is not in the header and must be inferred from the bytes in flight.

Tool question in the style of Q4 (traceroute). Related fact: traceroute sends UDP to unlikely high ports and the destination returns ICMP “port unreachable”, which is transport-layer demultiplexing at work.

3 Explain TCP flow control, including what happens when the receiver’s buffer is full.

The receiver allocates RcvBuffer and advertises **rwnd = RcvBuffer − (LastByteRcvd − LastByteRead)** in every segment. The sender keeps LastByteSent − LastByteAcked ≤ rwnd, so the buffer can never overflow even when the application reads slowly. If rwnd = 0, the sender stops, but it must keep sending **1-byte zero-window probes** (persist timer). Otherwise, if the receiver’s later window update were lost, both sides would wait forever. With the 16-bit field, the window can be at most 64 KB unless the **window scale** option is used.

3 Explain QUIC’s main features and compare its connection setup with TCP + TLS.

QUIC is an application-layer transport over UDP (used by HTTP/3). (1) **Connection setup**: TCP and TLS need 2 serial handshakes (2–3 RTT before data), while QUIC sets up transport and crypto state together in **1 RTT**, or 0 RTT when resuming. (2) **Streams**: multiple streams per connection with independent reliability and no HOL blocking, and **shared congestion control**. (3) **Always encrypted**, including most header fields, which improves privacy and blocks middlebox interference. (4) **Connection migration**: a connection ID instead of the 4-tuple, so a phone switching from Wi-Fi to cellular keeps the connection. (5) Loss detection and congestion control parallel TCP’s, and it runs in user space, so it evolves quickly.

4-mark questions

4 **cwnd trace. TCP Reno starts with cwnd = 1 MSS and ssthresh = 16. Loss is detected by 3 duplicate ACKs at the end of round 9, and by a timeout at the end of round 14. Tabulate cwnd for rounds 1–19 and state ssthresh after each loss. How many segments are sent in the first 5 rounds?**

Round	1	2	3	4	5	6	7	8	9	10
cwnd	1	2	4	8	16	17	18	19	20	10
Round	11	12	13	14	15	16	17	18	19	
cwnd	11	12	13	14	1	2	4	7	8	

Slow start: rounds 1–5, then CA. 3 duplicate ACKs at cwnd = 20: ssthresh = **10**, cwnd = 10 (Reno), then linear. Timeout at cwnd = 14: ssthresh = **7**, cwnd = 1, then slow start 1, 2, 4, 7 (capped at ssthresh), then linear.
Segments in the first 5 rounds = 1 + 2 + 4 + 8 + 16 = **31**.
Examiner will also accept cwnd = 13 in round 10 if you use the ssthresh + 3 fast-recovery rule. Say which one you are using.

4 **A client in India downloads a 10 MB file from a US server over a 100 Mbps bottleneck with RTT = 200 ms and MSS = 1460 B. Estimate the transfer time, identify the bottleneck, and suggest improvements.**

Ideal: $10 \times 8 \times 10^6 / 10^8$ = **0.8 s** of pure transmission time.

BDP = $10^8 \times 0.2 = 2 \times 10^7$ bits = 2.5 MB ≈ 1712 segments. The file is only about 4 BDPs, so **slow start dominates**.

Slow start from 1 MSS: reaching about 1712 segments takes about 11 RTTs ($2^{11} = 2048$), and about $2^{11} - 1 \approx 2047$ segments ≈ 3 MB have been sent by then. The remaining ≈ 7 MB at the full rate takes ≈ 0.56 s ≈ 3 RTT. With the handshake (1 RTT), the total is ≈ 15 RTT ≈ **3 s**, nearly 4 times the ideal.

If rwnd = 64 KB without window scaling: throughput ≤ 65,535 × 8 / 0.2 ≈ 2.6 Mbps, so the transfer takes ≈ **31 s**. **Flow control is the bottleneck.**

Improvements: enable window scaling and buffer auto-tuning, use a larger initial window (IW10), CUBIC or BBR for the high BDP, a CDN edge server close to the client (smaller RTT: see Q11 of your last paper), or several parallel connections (at the cost of fairness).

4 **Trace a TCP transfer in which one segment is lost. Host A sends five 100-byte segments starting at seq = 1000 (cwnd allows all 5). The segment with seq = 1100 is lost. Show every ACK, A's reaction, and the final ACK.**

A sends	B receives	B sends ACK
seq 1000	in order	1100 (or delayed)
seq 1100	**lost**	—
seq 1200	out of order, buffered	1100 (dup #1)
seq 1300	buffered	1100 (dup #2)
seq 1400	buffered	1100 (dup #3)
seq 1100 (fast retransmit)	fills the gap	**1500** (cumulative)

After the 3rd duplicate ACK, A retransmits 1100 **before its timer expires**. Reno sets ssthresh = cwnd/2 and cwnd = ssthresh (+3), then continues in CA. B delivers bytes 1000–1499 in order to the application. A single ACK of 1500 covers everything, so only **one** segment was retransmitted (GBN would have resent 1100–1400).

4 **An application must send sensor readings (small, frequent), firmware updates (large, must be exact) and live video. For each, choose TCP, UDP or QUIC and justify in terms of reliability, latency, congestion control and fairness.**

Sensor readings → UDP (possibly CoAP). Messages are small, and a handshake per reading would double the latency and the energy use. A lost reading is soon superseded by the next one. Keep the rate low, or add simple application-level ACKs for critical alarms.
Firmware update → TCP (or QUIC). Every byte must arrive exactly and in order. Connection setup is amortised over a large transfer, and congestion control keeps the transfer fair to other traffic. Integrity is protected end-to-end by the checksum plus an application hash or signature.
Live video → UDP-based (RTP/WebRTC) or QUIC. It is loss-tolerant but delay-sensitive, and TCP retransmissions and HOL blocking cause stalls. The application should still use its own **congestion/rate adaptation** (as DASH-style bitrate switching does) to remain TCP-friendly. QUIC gives per-stream delivery with no HOL blocking and built-in encryption.

Synthesis question that ties Chapter 3 back to the application layer, in the style of Q5, Q13 and Q17 from your last paper.

5 · Traps, and a last-hour checklist

Common traps

- The TCP ACK is the **next byte expected**, not the last byte received.
- Sequence numbers count **bytes**. SYN and FIN each consume 1 sequence number, and a pure ACK consumes 0.
- UDP demultiplexes on the **dest port only**; TCP uses the **4-tuple**.
- The routers along the path do **not** run TCP or UDP.
- TCP has **no NAKs** (rdt2.2 style): a duplicate ACK acts as a NAK.
- GBN has **one** timer and SR has one per packet. TCP has one timer but retransmits only one segment.
- Window limits: GBN $\leq 2^k - 1$; SR $\leq 2^{k-1}$.
- On **any** loss, ssthresh = cwnd/2. A **timeout** always sets cwnd = 1. Only **Reno** halves on 3 duplicate ACKs.
- The sending window is **min(cwnd, rwnd)**.
- Flow control ≠ congestion control. rwnd is in the header; cwnd is **never** transmitted.
- RTT = 2 × one-way propagation delay. Check whether L/R is ignored.
- The checksum uses 1's complement **with wraparound**; the receiver's sum should be all 1s.
- Neither TCP nor UDP guarantees delay or bandwidth.

Numbers to remember

α, β (EWMA)	0.125, 0.25
Timeout	Est + 4·Dev; doubles on each timeout
Delayed ACK	≤ 500 ms
Fast retransmit	3 duplicate ACKs
Headers	UDP 8 B, TCP 20 B, IPv4 20 B
Typical MSS	1460 B
Default RcvBuffer (slides)	4096 B
Max rwnd w/o scaling	65,535 B
Seq # / ACK #	32 bits; wraps at 4.29 GB
Avg AIMD throughput	0.75 W / RTT
Mathis	1.22·MSS / (RTT·√L)
ECN bits	2 in the IP ToS field; ECE/CWR in TCP
QUIC setup	1 RTT (0-RTT on resume)

Last-hour checklist: can you do each of these in under 3 minutes?

☐ List TCP vs UDP services and give 3 apps that use each, with reasons

☐ Explain what each of checksum, ACK, sequence number and timer fixes (rdt2.0 → 3.0)

☐ Compute stop-and-wait U and the window needed for a target U

☐ Find the sequence bits needed for GBN and SR for a given window

☐ Draw GBN and SR timelines with one lost packet

☐ Fill in seq/ACK numbers for a 2-segment exchange

☐ Do one EWMA update and compute the timeout

☐ Draw a 3-way handshake with x, y, x + 1, y + 1 and the socket calls

☐ Explain why not a 2-way handshake; the purpose of TIME_WAIT

☐ Explain SYN floods and SYN cookies; the listen backlog

☐ Compute rwnd; find the window-limited throughput

☐ Draw and label a cwnd graph (SS, CA, 3 dup ACKs, timeout)

☐ Compare Tahoe vs Reno; AIMD fairness diagram

☐ Compare CUBIC, BBR and ECN in one table

☐ Explain QUIC: 1-RTT setup, streams, no HOL blocking, encryption

☐ Explain what Wireshark can and cannot show for TCP vs QUIC

Exam technique that matched your last paper

- **“Compare” questions**: use a 3–4 row table (it earns marks quickly and the examiner can scan it).
- **“Trace / step by step” questions**: draw a two-column timeline with every seq and ACK value labelled, and state assumptions (loss position, whether L/R is ignored).
- **Numericals**: formula → substitution with units → boxed answer → one line of interpretation (“so the protocol, not the link, limits throughput”).
- **“Which is better?” questions** (like Q21): always name the condition, e.g. “SR is better *when loss is frequent or BDP is large*; GBN when simplicity and small buffers matter”.

Source material: Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Chapter 3 slides (v8.2). Extensions (SYN cookies, Karn’s algorithm, window scaling, PAWS, QUIC details) are standard RFC-level material; RFCs 793/9293, 5681, 6298, 7323, 3168 (ECN), 9000 (QUIC).